

Software Architecture Document (SAD)

Kruchten 4+1 View Model Architectural Description for VarsityConnect

PROJECT	DISCIPLINE	COMPLIANCE	STATUS
VarsityConnect	Senior Software Eng.	Kruchten 4+1 / ISO 42010	Approved & Implemented

Project: VarsityConnect Campus Community Platform

Course / Discipline: Senior Software Engineering (SSE)

Architectural Framework: Kruchten 4+1 View Model / ISO/IEC/IEEE 42010

System Version: 1.0.0

Status: Approved & Implemented

1. Introduction

1.1 Purpose

This Software Architecture Document (SAD) provides a comprehensive architectural overview of **VarsityConnect**, an integrated web-based campus community platform. It captures the critical architectural decisions, design patterns, subsystem decomposition, runtime processes, deployment topology, and cross-cutting concerns that govern the platform's implementation.

1.2 Scope

This document applies to the full lifecycle of the VarsityConnect platform, covering: - Modular Django applications: `registration`, `dashboard`, `course_material`, `marketplace`, `housing`, `messaging`, and core configuration in `SSE_PROJECT`. - Data persistence mechanisms (Relational database and media file subsystem). - Client-side responsive interface presentation and design system. - Verification and quality attribute evaluations.

1.3 Definitions, Acronyms, and Abbreviations

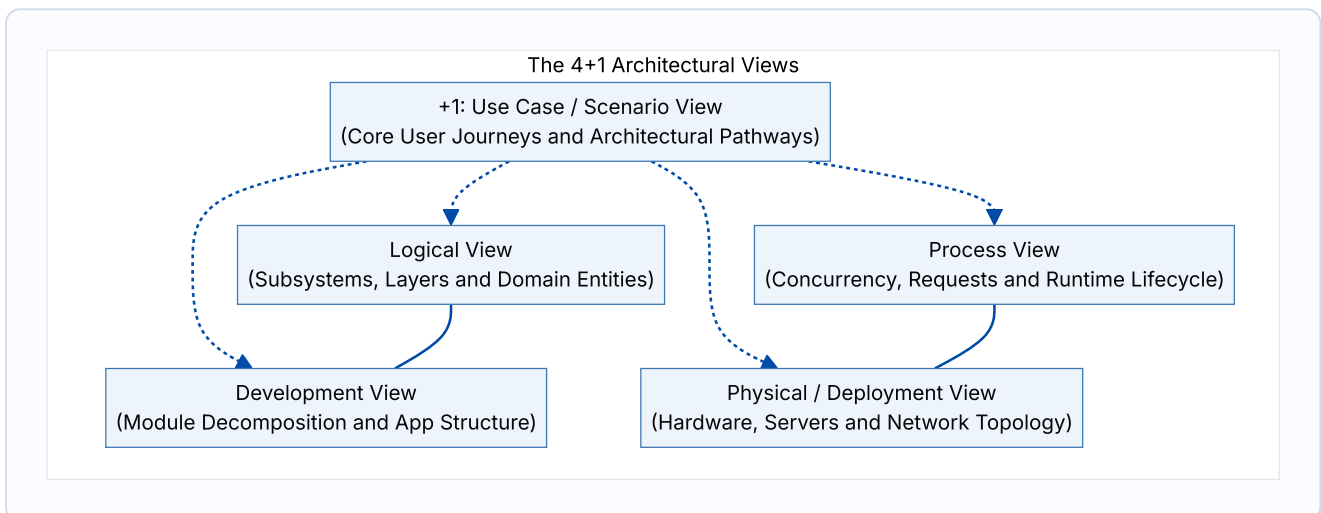
Term / Acronym	Definition
SAD	Software Architecture Document
MVT	Model-View-Template architectural pattern (Django standard)
WSGI	Web Server Gateway Interface (PEP 3333)
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
IDOR	Insecure Direct Object Reference
CSRF	Cross-Site Request Forgery
XSS	Cross-Site Scripting
P2P	Peer-to-Peer student transactions
TTFB	Time To First Byte

1.4 References

- Kruchten, P. (1995). *Architectural Blueprints—The "4+1" View Model of Software Architecture*. IEEE Software, 12(6), 42-50.
- ISO/IEC/IEEE 42010:2011: *Systems and software engineering — Architecture description*.
- Django Software Foundation. *Django Design Philosophies and Architecture Documentation*.

2. Architectural Representation (The 4+1 View Model)

To articulate the system architecture for various stakeholders (developers, system architects, and university examiners), this document adopts the Kruchten **4+1 View Model**:



1. **Use Case View (+1 View):** Grounded in key end-user scenarios illustrating how the system delivers functionality while validating architectural integrity.
 2. **Logical View:** Details the conceptual organization of the system, decomposing it into layers, subsystems, and domain classes.
 3. **Process View:** Addresses the runtime behavior, execution threads, request-response pipelines, and concurrency handling.
 4. **Development View:** Organizes the software modules, file packages, and dependencies within the developer environment.
 5. **Physical / Deployment View:** Maps the software components onto physical or virtual hardware nodes, network topologies, and hosting tiers.
-

3. Architectural Goals, Principles, and Constraints

3.1 Architectural Principles

1. **Model-View-Template (MVT) Pattern:** Strict separation of data structures (Models), business logic controllers (Views), and user presentation (Templates).
2. **High Cohesion and Loose Coupling:** Functionalities are encapsulated into self-contained Django applications (`course_material` , `marketplace` , `housing` , `messaging` , `registration` , `dashboard`) communicating via standard Python interfaces and relational keys.
3. **Defensive Security by Default:**
4. Universal authentication enforcement using `@login_required` .
5. Object-level ownership validation to eliminate Insecure Direct Object References (IDOR).
6. Mandatory CSRF verification for state-changing HTTP POST operations.
7. SQL injection immunity guaranteed through Django ORM parameterization.
8. **Design System Integrity:**
9. Adherence to the defined 5-color architectural palette:
 - **Steel Blue:** `#3e78b2`
 - **Cobalt Royal Blue:** `#004ba8`
 - **Slate Gray:** `#4a525a`
 - **Deep Charcoal:** `#24272b`
 - **Midnight Obsidian:** `#07070a`
10. Fluid typography and responsive design system ensuring zero clipping across mobile ($\leq 640\text{px}$), tablet, and desktop screens.

3.2 Key Constraints

- **Runtime:** Python 3.10+ execution environment with Django 6.x framework.
- **Database:** Relational ACID storage (SQLite for local rapid execution and testing; PostgreSQL for production deployment).
- **Session Policy:** Single-session browser-close expiration (`SESSION_EXPIRE_AT_BROWSER_CLOSE = True`).

- **File System:** Isolated media directory storage under `uploads/` with synchronized disk cleanup upon entity deletion.
-

4. Use Case View (+1 View: Key Scenarios)

The architecture is driven by four primary architectural scenarios:

Scenario 1: Academic Material Upload & Safe Binary Delivery

1. Student accesses `/course_material/`, completes the upload form with a PDF file and optional cover photo.
2. The view validates the multipart form, streams the files into `uploads/pdfs/` and `uploads/course_material/`, and writes the metadata to the database.
3. Another student views the material card. If the thumbnail image is absent or corrupt, an SVG document placeholder is automatically rendered.
4. Clicking "Download PDF" invokes `download_resource_view`, which streams the file as an attachment via `FileResponse` with sanitized file headers, preventing directory traversal.

Scenario 2: Peer-to-Peer Marketplace Transaction & Stock Tracking

1. Student authors a listing specifying category, condition, price, and inventory quantity.
2. Prospective buyers view the item card in a 3-column responsive grid and click into the listing detail page.
3. The detail page presents full seller metrics (rating, sales count) and an embedded "Contact Seller" CTA button.
4. The seller can adjust available inventory; decrementing quantity to zero switches the state pill from `In Stock` to `Currently sold out`.

Scenario 3: Real-Time Intra-Campus Communication & Soft Deletion

1. A buyer clicks "Contact Seller" from any listing.
2. The messaging engine checks for an existing conversation between the buyer and seller; if none exists, a new `Conversation` instance is created.
3. If the conversation was previously hidden by either participant, it is unhidden upon new message receipt.
4. When a user deletes a message or entire conversation, the system executes a non-destructive soft-delete by adding the user to the `hidden_for` relation, preserving the thread for the counterparty.

Scenario 4: User Onboarding & Dynamic Profile Aggregation

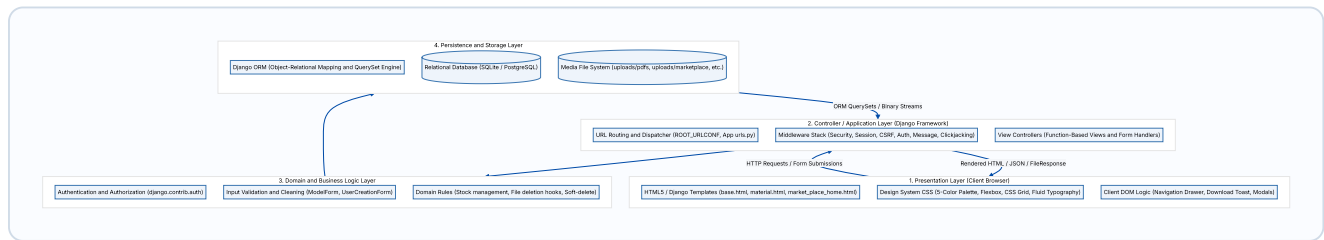
1. An unregistered student signs up via `registration_view`, where passwords pass validation rules.
2. The user is logged in, their session cookie is issued, and they are redirected to `/dashboard/`.
3. Navigating to `/profile/` triggers dynamic queries calculating the user's active marketplace listings, shared course notes, and posted jobs.

- The sidebar profile viewer dynamically renders the user's name with multi-line wrapping and ellipsis tooltip fallbacks to accommodate long names without visual clipping.

5. Logical View

5.1 Layered System Architecture

VarsityConnect follows a classic four-tier logical architecture:



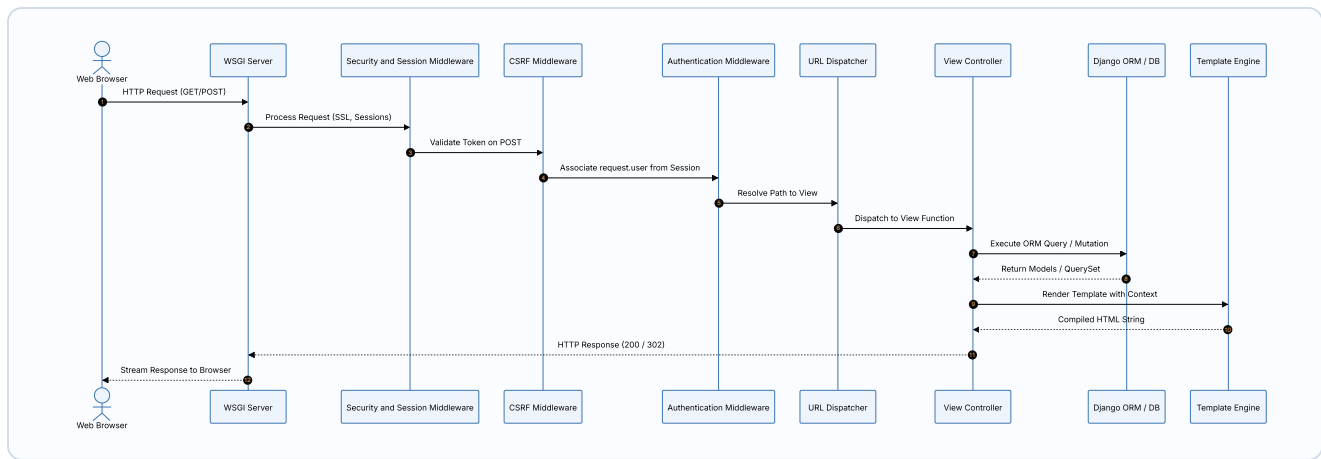
5.2 Subsystem Decomposition and Component Responsibilities

Subsystem / App	Primary Responsibilities	Key Components
registration	Account creation, credential authentication, session destruction, password policy enforcement.	landing_auth_view , registration_view , logout_thank_you_view
dashboard	Global metrics visualization, user profile page, cross-app metric aggregations.	home_dashboard_view , profile_view
course_material	Academic document indexing, file uploads, secure streaming downloads, physical file removal.	Resource model, ResourceUploadForm , course_material_view , download_resource_view , delete_resource_view
marketplace	Campus goods/services catalog, inventory tracking, seller information, item details.	MarketplaceListing model, MarketplaceListingForm , ListingStockForm , marketplace_home , listing_detail , update_listing_stock
housing	Job/internship postings, campus employment opportunities, accommodation listings.	Job model, Accommodation model, JobForm , job_list_view , job_detail_view
messaging	One-to-one messaging threads, message creation, conversation unhiding, soft-delete filtering.	Conversation model, Message model, inbox_view , chat_room_view , start_conversation_view , delete_message_view
SSE_PROJECT	Core settings, root URL dispatching, static/media serving routes, master HTML layout.	settings.py , urls.py , base.html

6. Process View (Runtime & Concurrency)

6.1 HTTP Request-Response Lifecycle

Each inbound HTTP request travels through a sequential middleware and view processing pipeline:



6.2 Concurrency and State Handling

- **Stateless Application Layer:** The Django application workers operate statelessly. Request context is dynamically populated using session keys stored in the database or cached backend.
- **Session Management:** Authenticated states are validated on every request via `AuthenticationMiddleware`. Sessions are tied to cryptographically secure signed session IDs in browser cookies.
- **Database Transactions:** Model mutations (`save()` , `delete()`) execute within atomic relational database transactions to prevent partial writes.

7. Development View (Code Organization)

7.1 Codebase Package Hierarchy

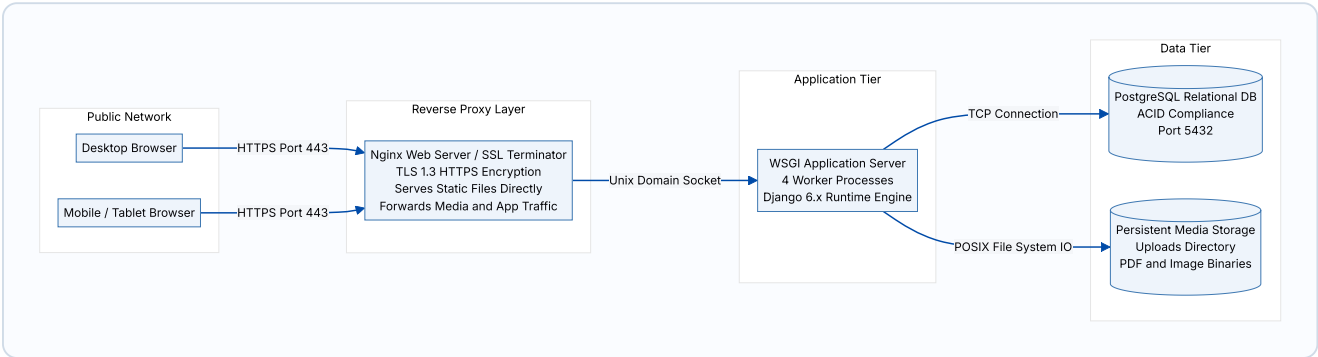
```
VARITYCONNECT/
├─ manage.py           # Django execution CLI
├─ db.sqlite3          # Local development database
├─ uploads/            # Uploaded media storage root
│   ├── course_material/ # Resource cover thumbnails
│   ├── pdfs/           # Uploaded academic PDF binaries
│   ├── marketplace/    # Marketplace item images
│   └─ accommodations/  # Housing photographs
├─ SSE_PROJECT/        # Core project configuration
│   ├── settings.py     # Global configuration & security settings
│   ├── urls.py         # Root URL routing & media serving
│   ├── wsgi.py         # WSGI entrypoint for web servers
│   └─ templates/
│       └─ base.html    # Master shell, sidebar, design tokens
├─ registration/       # Authentication subsystem
│   ├── views.py        # Login, Register, Logout views
│   └─ templates/registration/ # Auth HTML templates
├─ dashboard/          # Metrics and Profile subsystem
│   ├── views.py        # Dashboard & Profile logic
│   └─ templates/dashboard/ # Dashboard & Profile HTML templates
├─ course_material/    # Academic repository subsystem
│   ├── models.py       # Resource entity definition
│   ├── forms.py        # ResourceUploadForm
│   ├── views.py        # Upload, Download, Delete handlers
│   └─ templates/course_material/ # Grid layout & card components
├─ marketplace/        # Peer-to-peer commerce subsystem
│   ├── models.py       # MarketplaceListing entity definition
│   ├── forms.py        # MarketplaceListingForm, ListingStockForm
│   ├── views.py        # Home, Detail, Stock update, Delete
│   └─ templates/marketplace/ # Catalog & Detail HTML templates
├─ housing/            # Jobs & Housing subsystem
│   ├── models.py       # Job and Accommodation models
│   ├── forms.py        # JobForm
│   ├── views.py        # Job listings, detail, deletion
│   └─ templates/housing/ # Job board templates
└─ messaging/          # Intra-campus messaging subsystem
    ├── models.py       # Conversation & Message models
    ├── views.py        # Inbox, Chat Room, Start conversation
    └─ templates/messaging/ # Inbox list & Chat room templates
```

7.2 Model-View-Template Interaction Matrix

Subsystem	Model(s)	Primary View(s)	Primary Template(s)
registration	auth.User	landing_auth_view , registration_view , logout_thank_you_view	login.html , register.html , logout_thanks.html
dashboard	Resource , MarketplaceListing , Job	home_dashboard_view , profile_view	dashboard.html , profile.html
course_material	Resource	course_material_view , download_resource_view , delete_resource_view	material.html
marketplace	MarketplaceListing	marketplace_home , create_listing , listing_detail , update_listing_stock	market_place_home.html , create_listing.html , listing_detail.html
housing	Job , Accommodation	job_list_view , job_detail_view , delete_job_view	job_list.html , job_detail.html
messaging	Conversation , Message	inbox_view , start_conversation_view , chat_room_view , delete_message_view	inbox.html , chat_room.html

8. Physical & Deployment View

8.1 Deployment Topology



8.2 Environment Configurations

Attribute	Development Environment	Production Environment
Server Engine	Django Built-in Development Server (<code>runserver</code>)	Nginx Reverse Proxy + Gunicorn WSGI
Database	SQLite 3 (<code>db.sqlite3</code>)	PostgreSQL 15+ Server
DEBUG Mode	<code>DEBUG = True</code>	<code>DEBUG = False</code>
Security Headers	Local HTTP permitted	HSTS, Secure Cookies, TLS 1.3 enforced
Static & Media	Handled via Django <code>static.serve</code> routes	Offloaded directly to Nginx or S3 Object Storage

9. Data Architecture & Relational Mapping

9.1 Database Schema Organization

The database schema enforces referential integrity through foreign key constraints and cascade rules: -

`auth_user` : Central identity table containing authentication credentials, usernames, and system permissions. - `course_material_resource` : Stores academic material titles, categories, and paths to binary PDF files. Cascades on user deletion (`on_delete=models.CASCADE`). - `marketplace_marketplacelisting` : Manages item condition, pricing, inventory stock, and location metadata. - `housing_job` & `housing_accommodation` : Stores campus employment opportunities and housing listings. - `messaging_conversation` & `messaging_message` : Employs Many-to-Many join tables (`messaging_conversation_participants` , `messaging_conversation_hidden_for`) to support multi-user chat threads and individual soft deletions.

9.2 File Storage Synchronization Strategy

A critical architectural decision was made regarding file deletion: when an entity containing an attached file is deleted (e.g., `Resource` or `Marketplacelisting`), Django's default behavior removes only the database record, leaving physical files orphaned on disk.

To prevent disk saturation, VarsityConnect explicitly couples database entity removal with physical file deletion:

```
# Synchronized file deletion pattern implemented in views
resource = get_object_or_404(Resource, pk=pk, uploaded_by=request.user)
pdf_file = resource.pdf_file
image = resource.image
resource.delete()
pdf_file.delete(save=False)
if image:
    image.delete(save=False)
```

10. Cross-Cutting Concerns

10.1 Security Architecture

1. **Access Control:** Role-Based Access Control (RBAC) separates anonymous visitors from authenticated students and superusers. Object-level access checks ensure users can only modify their own items.
2. **Cryptographic Protection:** Password storage utilizes PBKDF2 with SHA-256 password hashing. Session IDs are cryptographically random 128-bit tokens.
3. **Cross-Site Request Forgery (CSRF):** Synchronizer Token Pattern implemented across all POST forms.
4. **Content Security:** Uploaded files are served with explicit MIME types (`application/pdf`) and sanitized filenames to avert header injection and path traversal attacks.

10.2 Responsive UI & Design Tokens

The user interface is governed by a unified CSS design system anchored in `base.html` : - **CSS Custom Properties (Variables):** - `--c-steel: #3e78b2` - `--c-cobalt: #004ba8` - `--c-slate: #4a525a` - `--c-charcoal: #24272b` - `--c-midnight: #07070a` - **Fluid Layouts:** Uses CSS Grid (`repeat(auto-fill, minmax(...))`) and Flexbox to automatically reflow content. - **Sidebar Profile Resilience:** Increased sidebar width to 280px with 2-line clamp (`-webkit-line-clamp: 2`) and `overflow-wrap: break-word` to accommodate multi-word student names without clipping.

10.3 Media Fallback Subsystem

To guarantee platform visual polish and reliability: - Listing and resource cards utilize native `<img>` tags accompanied by an inline `onerror` fallback handler. - If an image is absent, deleted, or fails network retrieval, the card automatically displays a stylized SVG placeholder on an obsidian gradient surface with category text, preventing empty white void anomalies.

11. Architectural Evaluation & Trade-off Analysis

Decision / Alternative	Selected Approach	Trade-offs & Justification
Architecture Pattern: SPA (React/Vue) vs. Server-Side MVT (Django)	Server-Side MVT	Benefits: Eliminates client-side build pipeline complexity, native SEO and security token injection, faster initial load, unified Python codebase. Trade-off: Page transitions require standard HTTP navigation.
Real-time Messaging: WebSockets (Channels) vs. Synchronous HTTP Polling	Synchronous HTTP Messaging	Benefits: Zero additional daemon dependencies (Redis, Daphne), simplified deployment, ultra-reliable delivery. Trade-off: Message refresh requires user action or lightweight HTTP poll.
Message Deletion: Hard DB Deletion vs. Soft Deletion (<code>hidden_for</code>)	Soft Deletion (<code>hidden_for</code>)	Benefits: Prevents one party from destroying legal/audit transaction records of another party in peer commerce. Trade-off: Requires filtering queries with <code>.exclude(hidden_for=user)</code> .
Image Presentation: CSS Background Images vs. Semantic <code></code> + SVG Fallbacks	Semantic <code></code> + SVG Fallbacks	Benefits: Accessible to screen readers, responsive <code>object-fit: cover</code> , robust <code>onerror</code> recovery, prevents unstyled empty card voids. Trade-off: Requires slightly more template markup.